

# 基于几何锚定的视觉大模型与轻量模型协同的复杂金融文档还原方案

AFAC2026 挑战组赛题二·方法描述

队伍 noone ·杨康、赖坤豪

2026-08

## 1. 任务与核心判断

赛题要求把超长/超大金融文档截图还原为高保真 Markdown，评分由文本编辑距离、表格 TEDS、阅读流编辑距离三项等权构成。唯一可用的视觉语言模型是主办方提供的 FinixDoc-VL API，不可微调、不可替换。

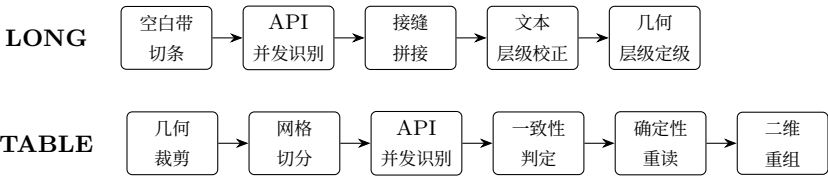
方案设计的前提是先摸清这个黑盒的行为边界。正式开发前用标定实验测得四条硬约束：

| 特性      | 实测结论                    | 对方案的约束                  |
|---------|-------------------------|-------------------------|
| 接口形态    | 只接受图片，无 prompt 等文本参数    | <b>结构控制只能在客户端</b>       |
| 单次输出上限  | 约 10–12k 字符，与图像尺寸无关     | 调用次数由 <b>内容量</b> 而非像素决定 |
| 内部分辨率上限 | 约 1500 px，更大的图被内部降采样    | 输入须切到 1500 px 内，否则糊字    |
| 并发能力    | 16 路并发无限流，延迟仅 10s → 13s | 时间非瓶颈，可细粒度切分切换精度        |

其中内部分辨率上限最具决定性：任何保留全宽的切分都无效。由此确立本方案的核心原则，**把结构决策权从模型手中收回到代码中**：几何负责定位（切在哪里、有几行几列、哪些位置为空），模型只回答“这个位置是什么内容”。其意义在于错误的**可能性边界**：模型自回归解码在高度重复内容上会丢失行列追踪（跳行、复读、展平），容错只能降低概率；而当行列由代码给定、模型每次只回答单格内容时，“数错行数”这类错误的发生条件本身不存在。

## 2. 系统总览

两类图的物理形态与失败模式完全不同，按官方发布目录分流到两条独立流水线（不做长宽比自动分类，以免平白引入一类错误）：LONG 面条图宽度锁定约 1500 px、恰在模型有效分辨率内，可整宽送入、仅需纵向切分；TABLE 大表图宽达两万余像素、必须二维切分。TABLE 的 GT 内容量中位约为 LONG 的数倍（按每次调用 10k 字符折算，50 张 TABLE 合计约 694 次调用、50 张 LONG 仅约 89 次），是计算量与难度的双重主体。两条流水线各自产出 Markdown/HTML，合并后按 file\_name 排序写出 submission.csv。



（对应模块：LONG 为 slicer\_long / api\_client / stitch\_long / heading\_norm / geom\_heading；TABLE 为 crop / slicer\_table / api\_client / grid\_ocr / cell\_ocr / stitch\_single。）图级并行用进程池（绕过 GIL，几何计算 CPU 密集），图内 API 调用用线程池并发，单图异常不影响整批。

## 3. TABLE 流水线

TABLE 路是本方案的主体。整体分三个阶段：Stage I 用纯几何手段把整图分解为若干**单一子表**；Stage II 在每个子表内按网格线切分并识别，同时对每个切块做一致性校验与修复；Stage III 把校验通过的切块按全局坐标重组为完整表格。

### 3.1 Stage I: 几何裁剪与子表切分

**动机来自评测口径。**官方 TEDS 按出现顺序对 GT 与预测中的 `<table>` 逐个配对。若一张图上有  $N$  个并列子表而系统将其读成一个大表，则只有 GT 的第一个表能被配上，其余全部计零分。**输出的表数量必须精确等于 GT，差一个就会造成全盘错位。**

这一阶段完全不调用模型，输出是一个有序的块序列  $[(k_i, b_i)]$ ，其中  $k_i \in \{\text{text}, \text{table}\}$  为块类型， $b_i$  为包围盒。分三层：

**第一层，剥离表外文字。**页眉、页脚、水印、页码等元素若混入表格区域会污染网格估计。判据是三个条件的合取：块高度低于阈值（矮）、块宽度显著小于主体（窄）、且非最大连通分量。三者缺一不可，“窄”这一条尤其关键，它保护了三角形稀疏表底部那些又宽又矮的有效数据段不被误剥。

**第二层，切分并排子表。**采用两条几何判据的合取：

判据一，缝高倍数。定义整行墨量低于宽度 0.3% 的像素行为白线，连续白线构成白带。设白带高度的中位数为行缝基准  $h_0$ ，则真子表缝需满足

$$h_{\text{seam}} \geq k \cdot h_0, \quad k = 2$$

$k$  的取值有明确的数据依据：实测真子表缝为行缝的 2.5–10 倍，单表内空行造成的缝约为 1 倍，两个分布之间在  $[1.1, 2.5]$  区间内几乎没有样本， $k = 2$  落在这个空隙中央。此处不使用 Otsu 等自适应阈值，单表内所有行缝等高时 Otsu 会退化，把每一条行缝都当作切点。

判据二，框线连续性。仅靠墨量无法区分“表内空行”与“真子表边界”：稀疏表底部的空行虽然墨量为零，但表格的边框与列线仍连续穿过。因此要求缝内贯穿的竖线条数不超过 2。实测真子表缝处框线断开（贯穿数为 0），而表内空行处仍有多条竖线贯穿。

**第三层，标题段归类。**子表上方的标题文字被空行隔开，形成前导矮段。这类段必须单独标记为 text 类型，任何送入表格识别流程的块都会被包装成一个 `<table>`，标题段会凭空产生一个空表，使表数量加一，触发前述的全盘错位。标题段与表外文字共用同一套纯文本识别路径。

### 3.2 Stage II: 网格切分

在单个子表内，切分必须严格落在检测到的网格线上，使每个切块 (tile) 包含**整数行、整数列**。这条约束直接消除两类密表病态：切点落在行内会导致边缘半行的取舍在相邻块之间不一致（行数相差一行）；又宽又矮的薄条会使模型无法辨识行边界，把整块内容压进单个 `<tr>`（曾观察到单块输出 1 行  $\times$  819 列的极端展平）。

检测到的行列骨架在切分前经两步修复：**漏检列线修复**，若一个宽列内墨迹呈双簇、且各数据行的簇间空隙跨行对齐，说明两列被并读，结合局部识别证据在缝心插入列界；**幻影窄列清理**，若某列宽度显著小于中位列宽、且几乎无独立墨迹（多为跨列文字或残余线条），删除其边界、把窄条并回相邻真实列。若骨架仍严重错位（局部列带的行结构与全表明显不一致），则不在错误坐标系上强行装配，而是整体转入自由分块识别的回退路线，一张极少触发的安全网。

切块尺寸上限为 25 行  $\times$  15 列，由单变量隔离实验确定。

**空白块预判。**墨量极低的切块不送模型，直接按已知网格填充空单元格。这既节省调用（100 张图上省下约 1125 次），也根除了一类幻觉，由于无法用提示词告知模型“此处为空，请勿编造”，唯一可靠的做法是不把空白送进去。

**小字上采样。**定义切块的等效格边长

$$e = \sqrt{\frac{A_{\text{eff}}}{n_r \cdot n_c}}$$

其中  $A_{\text{eff}}$  为去除页边距后的面积， $n_r, n_c$  为行列数。当  $e < 40$  px 时判为密集小字，放大倍数取

$$s = \text{clamp}\left(\frac{75}{e}, 1, 3\right)$$

即把格边长调整到目标值 75 px。使用二维格面积而非一维列间距，是为了正确识别“列稀行密”型表格。

这里有一个反直觉但可复现的实测结论：**密集小字块在原始分辨率下，模型不仅识别更差，速度也更慢**，单块耗时 181 秒且输出行幻觉与列漂移，上采样 2.58 倍后仅需 45 秒且识别正确。合理解释是模型在超出其有效分辨率的输入上会反复挣扎、走入慢速解码路径。因此上采样并非“以时间换精度”的取舍，而是精度与延迟的同时改善。

### 3.3 一致性判定与确定性重读

这是 TABLE 路的核心机制，也是最终版本相对前代的主要增益来源。设计遵循三段式：**几何期望** → **一致性判定** → **确定性重读**。

**几何期望**。对每个切块，由墨迹几何独立计算出一个期望结构，不依赖模型输出。设  $\text{ink}(i, j)$  与  $\text{gray}(i, j)$  分别为严阈值与松阈值下格  $(i, j)$  的墨迹指示（严阈值取灰度 128，捕获正文浓墨；松阈值取 180，兼容淡灰框线与浅色标题），定义格占位

$$\text{cell}(i, j) = \text{ink}(i, j) \vee \text{gray}(i, j)$$

期望结构  $E$  即由全部  $\text{cell}(i, j)$  构成的二值占位图：哪些行含有内容、每行的哪些格有墨。

**一致性判定**。模型返回的解析结果需满足**占位恒等**：非空行数等于  $|E|$ ，且逐格满足

$$\forall(i, j) : \text{非空}(i, j) \iff \text{cell}(i, j)$$

判定为零容差，不允许众数表决，不允许  $\pm 1$  行的容差。判据必须做到逐格：仅约束列数上界或每行宽度，都只是占位图的一维投影，会给“重复输出藏身其间”或“中段占位缺失、内容整体错位”留下不被察觉的窗口。仍不可见的残余情形是“占位完全相同、仅数值被读错”，需内容层面的数值平滑性验证才能捕获，属未来工作。

**确定性重读**。判定不通过的切块触发唯一一个修复动作：整块回落本地 PP-OCrv6s 逐格识别，对期望占位为真的每个格，按几何行列边界裁剪、放大后本地识别（结果按内容哈希缓存），期望为假的格置空，再由几何骨架装配成规整网格。行列位置完全由骨架给定，本地模型只回答“这个格子里是什么”，计数类错误（行数爆炸、漏行、跳读）在结构上不可能发生。单调重复区恰恰是 API 最易失稳、而逐格识别最易处理的输入，本地模型输出确定、重放稳定。这一通用修复动作替换了前代针对各类病态分别编写的七个专用修复模块，代码减少约 370 行而覆盖更完整，判据越严格，抢救逻辑越少。

### 3.4 Stage III：二维重组

重组把各切块的识别结果按全局行列坐标装配为完整表格，并处理以下结构问题：

**全局列宽一致**。各横带独立重组时，若真实块使用模型返回的列宽、空白块使用几何检测的列宽，二者不一致会导致整表参差不齐，TEDS 崩溃。解决办法是为每个切块列  $c$  确定一个规范列宽  $W_c$ （该列各横带非退化块列宽的众数），所有横带的第  $c$  块统一对齐到  $W_c$ ，使每行的总列数恒为  $\sum_c W_c$ 。

**跨列表头的 colspan 合成**。表头行中的文字往往横跨多个数据列。按内容簇分段：每个文字格向右吞并连续空档直到下一个文字格，行首空档并入首格，生成对应的 colspan 属性。这里设有**数据行守卫**，训练集中带尾部空单元格的首行全部是以数字为主的数据行，GT 保留空 `<td>` 而非合成 colspan；因此当非空格中数字格占多数时不做整形。

**表头结构重建**。以“列号行”为锚点：在前三行中寻找连续整数游程长度不小于 5 的行，其上一行按内容形态分三种情况处理，单个短簇折入下方角格形成斜线表头，全为短簇则识别为双行表头（左侧标签上下合并为 rowspan，右侧标签簇发 colspan），含长簇（超过 10 字）则整行提升为表格标题。列号行以上的所有行一律标题化。

**编号序列补齐。**表格中的行号列与列号行常因识别丢失而出现空洞。按局部连续段做线性补齐（横向限前 3 行、纵向限前 4 列，仅补 0–9 范围）。必须按局部段而非全局仿射拟合，并排双面板的两段列号行截距不同，全局拟合会同时算错两段。

**碎片回收。**Stage I 误剥为标题的表头或数据行，在识别阶段被判定为表格碎片后，Stage III 优先拼回紧随其后的表顶；若无可合并的下方表格，则降级为普通文本，避免产生孤立的小假表。

## 4. LONG 流水线

LONG 路的结构问题远轻于 TABLE：面条图宽度锁定 1500 px，恰好落在模型的有效分辨率内，横向不需要切分。核心挑战是切点选择、接缝处理与标题层级。

### 4.1 自适应切分

不做等高裸切，而是求每行墨量的水平投影，在**行间空白带**下刀，使切点落在文字行之间。这直接消除了接缝处的半行残字，后者是阅读流与文本编辑距离的主要失分源。切分只沿**高度方向**进行，目标条高 5000 px：宽度已锁定在 1500 px（模型有效分辨率上限，见第 1 节）之内、无需横切，故 5000 px 的切高不受该分辨率上限约束；其依据是实测延迟随高度近似线性增长（1500 px 约 8 秒，5000 px 约 13 秒，10000 px 约 27 秒），5000 px 是延迟与条数的平衡点。

**表格避让。**条带内若检测到表格，切点应绕开，避免把一张表切成两半。表格检测按“线”而非按像素列计数（一条框线通常占 2 个相邻像素列，按列计数会高估一倍），且逐带独立核验，要求贯穿竖线不少于 3 条（左边框 + 列线 + 右边框），阈值取 2 会把双栏排版的分栏线误判为表格。

避让规则附带一条重要约束：**带高超过目标条高的表不避让**，对本就放不下的大表，避让买不到完整性，反而顶高切点、造出异常短条带并引发连锁位移。

### 4.2 接缝拼接

拼接需处理两类情形：

**普通接缝。**相邻条带以不重叠的坐标裁出，正文接缝只需续行、不做模糊去重（模糊去重的相似度会被空行与套话抬高、误删正文），表内的重叠由专门路径处理。

**跨条表格。**长表格中部没有空白带，切点必然落在表内，导致上一条被收口为 `</table>`、下一条重新开启 `<table>`。此处需要跨条重连。另有一类情形是切点穿过某个长单元格，使该格的前半部分被读到表外形成孤儿文本，识别条件为：上条已闭表、其后的孤儿文本很短且以连接标点结尾、两侧表格列数相同，满足时将孤儿前缀塞回下条首行的末单元格。

### 4.3 标题层级：文本相对定级与几何绝对定级

模型按条带识别，# 的层级是各条带内的局部判断，拼成整篇后同一编号序列常在条带边界被重置。层级处理分两级，先文本后几何。

**文本相对定级**（heading\_norm）采用栈模型：维护一个大纲栈，按编号格式的连续性判断当前标题是同级、下钻还是回到上级。关键修正是，当遇到一个此前出现过的编号序列时，采用该序列的**历史层级**，而非相邻标题层级加减一。后者会在“深层游走后回到浅层”的情形下持续漂移。

此外还有三条针对性处理：约 56% 的文档中，封面大标题被模型拆成若干无 # 的普通段落，需合并并提升为一级标题；目录条日常被输出为 \* 1.1 ... 形式的列表项，需在目录区转为标题；表内满宽横幅行的 colspan 需归一化（训练集 GT 中该写法有 102 例，写作裸 `<td>` 的仅 5 例）。

**几何绝对定级**（geom\_heading）是 LONG 路的最后一步，也是全流程中唯一回到原图做全局裁决的一步。标题层级本质上是排版信息，前述所有步骤都只能从文本形态间接推断。这一步回到原图逐行测量：

- **字号**：取行内 core 高度（墨量不低于中位数 0.6 倍的像素行数），相对本文正文中位数归一，每个文档独立计算；
- **淡横线**：检测标题上下的浅色分隔线。

裁决规则是：淡横线上方判为章级一级标题；无编号的中段标题按字号判定，不低于正文中位数 1.25 倍者保留为一级标题，否则判为模型误报并删除该标题标记。逐行识别使用同一个本地小模型，结果按图片名缓存。

这一步必须放在拼接与文本定级之后，只有整篇拼合完成，字号才具有全局可比性。

### 5. 参数设置

| 参数                             | 值                            | 依据                                                  |
|--------------------------------|------------------------------|-----------------------------------------------------|
| MAX_CONCURRENCY                | 32                           | 图内 API 调用的线程并发数                                     |
| POOL_PROCS                     | 6                            | 图级并行进程数（几何计算 CPU 密集，进程池绕开 GIL）                      |
| API_TIMEOUT                    | 40 s                         | 单次请求超时；服务端过载时“收下不回”，久等无收益                           |
| API_RETRIES                    | 3                            | 失败重试次数，失败项由缓存重跑补齐                                   |
| DEGEN_RETRIES                  | 1                            | 复读退化的额外重试；退化是确定性的，多试无收益                             |
| LONG target_h                  | 5000 px                      | 延迟随高度线性增长，此处为延迟与条数的平衡点                              |
| LONG MIN_RULES                 | 3 条                          | 表格至少含左框、列线、右框；取 2 会把双栏分栏线误判成表                       |
| TABLE 切块上限                     | 25 行 × 15 列                  | 单变量隔离实验确定                                           |
| 上采样阈值 / 上限                     | $e < 40\text{ px} / 3\times$ | 目标格边长 75 px；二维格面积可识别“列稀行密”型表格                       |
| 子表缝高倍数 $k$                     | 2                            | 真缝为行缝的 2.5–10 倍、空行缝约 1 倍， $k = 2$ 落在两分布的空隙中央        |
| 缝内贯穿竖线上限                       | 2 条                          | 真子表缝处框线断开，表内空行处框线仍连续贯穿                              |
| BIN_INK / BIN_FAINT / BIN_LINE | 128 / 180 / 170              | 严档 (128) 捕正文浓墨、松档 (180) 含淡灰框线与浅标题；170 专供并排面板中缝与横线检测 |

全部阈值均为几何或统计维度上的通用判据，不含针对特定图像的分支或白名单。

### 6. 迭代历程

TABLE 路经历了三次架构级迭代，LONG 路的演进相对平缓。以下分别叙述。线上分数按分半提交口径给出（TABLE 与 LONG 分别提交、另一半置空，因而可分别归因）。

#### 6.1 TABLE 第一代：整表切分（约 80 分）

第一代把每张图当作单个表格处理：检测网格线，按网格切分为切块，逐块识别后按坐标拼回。

这一代的起点是一个决定性的实验结论。最初的行带切分（只切高度、保留全宽）在中位表上切了 16 个带，TEDS 仅 0.49，内容只读回 62k/74k 字符，原因正是宽度超过模型内分辨率上限而被降采样。改为原生分辨率的二维方块切分后，内容召回率的对比如下：

| 切块尺寸           | 中位表切块数    | 内容召回率        |
|----------------|-----------|--------------|
| 1100 px        | 35        | 0.979        |
| <b>1500 px</b> | <b>20</b> | <b>0.998</b> |
| 2000 px        | 12        | 0.920        |
| 2400 px        | 6         | 0.808        |

1500 px 锁定为切块上限。至此问题从“内容读不回来”转化为“读回来的碎块如何拼成正确结构”。

在这一架构上我们做了十余轮改进：多子表分段（利用模型自身返回的多个 `<table>`）、列数封顶、超大切块强制细分、截断表恢复（补齐未闭合标签而非整块丢弃）、展平幻觉过滤、空白判据阈值调整、全局列宽一致、尾部全空列裁剪等。本地全量交叉验证的综合分从 71.1 提升至 83.2，线上 A 榜达到 80.44（其中密集小字上采样是单项最大增益）。

**瓶颈与放弃的原因。**到 83 分附近，改进变得越来越困难且互相冲突，修好一类表往往打坏另一类。根本原因是这一代**没有可靠的行列估计**：所有结构决策都依赖模型返回的行列数，而模型在密表上的行列分割本身就不稳定（同一横带的各列块可能读出 11、12、0 等互不相同的行数）。在一个不可靠的基准上做修补，只能不断增加特例分支，越改越乱。期间也验证过用检测到的网格骨架强制约束模型行数，但当时检测精度不足（83 列对真值 70），硬约束反而把原本正确的切块也搅乱（均值 0.38  $\rightarrow$  0.09），由此确立“检测结果只作软先验、不作硬约束”的原则。

## 6.2 TABLE 第二代：全面几何化（约 92 分：TABLE 52 + LONG 40）

第二代是架构重写，核心是把结构决策从模型转移到几何：

1. **剥离全部表外文字**：页眉、页脚、水印、页码，以及子表标题，一律先切出来单独走纯文本识别；
2. **子表切分**：用缝高与框线连续性两条判据把整图切成若干单一子表，每次只处理一个（此前试过固定间隙、覆盖度、自适应、降采样问模型、列指纹五种检测均不稳健，根因是三角表的自然收窄与真子表边界在投影上不可区分，过切会破坏当时已达 0.7–0.95 的 74 张单表）；
3. **行列估计**：在单个子表内独立完成行列网格的几何估计，作为后续所有装配的坐标基准。

第 2 项的效果最为显著：整体本地分从 84.07 提升至 88.37，其中被切分的多子表从 69.66 提升至 83.94（提升 14.28），而未被切分的单表输出逐字不变，这是一个零风险的改动，因为判据不满足时流程完全退化为原路径。

行列估计的精度在这一代达到可用水平：有框表的列估计 51/52 精确，无框表的行估计精确率从 77/113 提升至 106/113、列估计从 23/96 提升至 84/96 且零多估。

**残留问题。**即使行列估计正确，模型内容仍可能失配，形态多样：幻读、展平、少行、少列。第二代针对每种形态分别编写修复逻辑，对半重读、截断补救、展平三路抢救、塌缩拆格等共七个模块。这些模块彼此耦合、触发条件互相干扰，且各自只覆盖已知形态，遇新形态即失效。

## 6.3 TABLE 第三代：统一的一致性判定与确定性重读（TABLE 53.34）

第三代的洞察是：上述七种失配形态是**同一个问题的不同表现**，模型返回的结构与几何期望不一致。既然如此，就不需要分别识别每种形态，只需要一个判据判断“是否一致”，和一个动作处理“不一致”。

于是有了 3.3 节的三段式：几何期望给出零容差的占位恒等判定，不通过则整块回落本地小模型逐格重读。七个专用修复模块全部删除，代码减少约 370 行。

线上 TABLE 半边达到 **53.34 / 55**，平均 TEDS 约 97%。在 B 榜集合上，223 个切块触发了本地重读，残余未解决的审计报告警仅 4 条。

## 6.4 LONG 路的演进

LONG 路的基础流水线（空白带切分 + 接缝拼接）在早期就达到了较高水平，本地综合分约 92.5 且长期无回归。后续改动集中在两处：

**标题处理。**依次实现了栈模型相对定级、封面标题合并提升、目录列表项转标题、几何绝对定级。一次隔离提交证明**层级数字本身不计分**（仅改 627 个标题的层级、正文一字未改，线上分毫无变化）；实际增益来自那些改变文本内容的操作，合并被拆散的封面标题、目录项转标题、删除误报标题。据此把 LONG 的投入转向真正的失分源：残余失配约 41.4% 是形近字、29.4% 是断行位置差异。

**长文内表格。**面条图中散布着表格，而表格在评分中权重较高。这方面的改动包括表格避让切点、跨条表格重连、孤儿单元格回收、满宽横幅行的 colspan 归一。这一批改动使线上 LONG 半边从 40.49 提升至 **41.99 / 45**，训练集上的表格 TEDS 从 95.27 提升至 96.69，方向一致。

## 7. 推理资源与时间

不依赖 GPU。全流程为 CPU 计算、远端 API 调用，以及一个 5.27M 参数的本地 CPU 小模型。

| 项  | 配置                                                            |
|----|---------------------------------------------------------------|
| 硬件 | 24 核 CPU / 62 GB 内存 / 无 GPU；建议不低于 8 核、16 GB                   |
| 软件 | Python 3.11 / Linux；依赖见 <code>requirements.txt</code> ，无需编译扩展 |
| 磁盘 | 不低于 2 GB（API 结果与本地识别结果的落盘缓存）                                  |
| 并发 | 图级 6 进程 × 图内 32 线程                                            |

**推理时间：**B 榜实测两个半边各约 10 分钟（各 50 张，并发 32）。引入本地小模型承担残差重读与几何定级后两侧均略有增加，几何定级冷启动约 5 秒/张，残差重读仅在判定不通过的切块上触发。全量 100 张按 30 分钟预估，留足余量亦在 1 小时以内，远低于 3 小时上限。

耗时的主要变数在 API 侧负载而非本地计算，这也是 `API_TIMEOUT` 设为 40 秒快速放弃、依靠缓存重跑补齐失败项的原因。

**未使用 ensemble。**全流程只有 FinixDoc-VL 一个视觉语言模型，无多模型投票、无多次采样融合。本地小模型不参与投票，仅在一致性判定不通过时作为确定性执行体接管，二者是串行的职责划分而非并行的结果融合。全部增益来自几何前处理与结构后处理。

## 8. 主要创新点

**表格感知的自适应切分。**面条图沿行间空白带下刀、密表严格按几何网格线切出整数行列的二维块，切点永不落在文字行或单元格内部。好处是从根本上消除接缝处的半行残字与密表相邻块的行数抖动，这两类正是常规等高裸切的主要失分源。

**原生分辨率的自适应上采样。**按切块的等效格边长估计文字密度，对密集小字块放大到目标格边长后再送识别。好处是双赢：既避免超过模型有效分辨率的输入被内部降采样糊掉小字，又反直觉地缩短了模型解码延迟，密集小字块在原图分辨率下既慢又错、上采样后既快又准，无需在精度与延迟间取舍。

**几何骨架的自适应行列估计。**由墨迹几何独立估出行列骨架（含漏检列线修复、幻影窄列清理），不依赖模型返回的行列数。好处是把结构决策权从模型收回代码：模型退化为无状态的单格问答，行数爆炸、漏行、整块展平这类计数错误的发生条件本身不复存在，这是与“增加容错、冗余投票”等常规纠错的根本区别。

**用几何占位判断 API 对错的零容差校验。**从原图墨迹算出每格“应有内容 / 应空”的期望，要求模型输出逐格恒等匹配，否则整块判不合格。好处是不依赖模型置信度、以原图物理证据当质量门，能统一发现幻读、漏行、错位、展平；且判据越严、抢救逻辑越少，七个专用修复模块被一个通用动作取代，越严越简。

**本地小模型的确定性兜底。**不合格块整块回落本地 PP-OCRv6s 逐格识别，行列位置由骨架给定、模型只回答单格内容。好处是单调重复区（API 最易失稳的输入）恰是逐格识别最稳的场景，输出确定、可复现；且只在少数不合格块触发，CPU 成本可控。

**空白块预判绕开编造幻觉。**因接口无法用提示词告知“此处为空”，改用墨量投影自行预判空白块、直接按骨架填空而不送模型。好处是既省下大量调用，又根除了模型对空单元格的编造，一个由接口约束反向催生、比“能用提示词”更鲁棒的前处理。

## 9. 局限与展望

**局限。**本方案的几何判据主要面向印刷体、规则或半规则的金融表格，面对更复杂多样的表格仍有局限：无框表的行列骨架仅靠墨迹推断，遇到列距不均、稀疏交错的版式时精度会下降；跨页延续的表格、图文混排、旋转或透视变形的截图、手写与彩色底纹表格尚未专门处理；几何骨架一旦估计错误，误差会传递到后续的局部识别。残余失分则主要在识别层面（约 41% 形近字、29% 断行位置），属内容细节而非结构问题。

**展望。**可引入 10M 参数预算内的轻量版面模型提升复杂版式的鲁棒性，为切块注入表头与标签列锚点，以数值平滑

性等结构先验做离群校验。更一般地，本方案“几何定位 + 模型识别 + 零容差校验 + 确定性兜底”的模式不限于文档解析：任何“大模型在长序列上丢失位置追踪”的任务（长表格生成、长列表处理、多轮结构化抽取）都可沿用，把位置控制权交还代码、模型只做稳定的局部判断。

## 10. 合规声明

- **大模型使用**：全工程唯一的视觉语言模型接口是主办方提供的 FinixDoc-VL API，调用点集中于 `src/common/api_client.py`（两处 `requests.post`），无任何其它第三方模型 API、SDK 或网络地址。
- **提示词**：该 API 请求体仅含 `userId / apiKey / fileName` + 图片文件，不接受文本提示词参数，因此本方案不存在提示词配置文件。
- **本地小模型**：PP-0CRv6\_rec\_small.onnx，**5.27M 参数**（低于 10M 上限），CPU + onnxruntime 推理。同一引擎初始化时载入的检测与方向分类模型在调用时均置 `use_det=False / use_cls=False`，不参与推理；三者参数量合计亦仅 7.72M。模型文件随 rapidocr 安装包一并提供，运行期不联网下载。
- **无结果硬编码**：代码中不含针对特定测试图像的固定输出、白名单或标识符分支。
- **一键复现**：`bash run.sh` 单条命令产出完整的 `submission.csv`。